DA03 Arithmetic, revenue and reading errors

Week 1, Meet the data | Session 3 of 24 | Python for Data Analysis

Today's question. How much money did the first three sales bring in?

Objective. By the end: you can calculate revenue for a sale, check it by hand, and read an error message to find the line to fix.

> **WHERE THIS IS USED**
> Revenue is units times price, added up. A shop owner does it by hand, and a program does it for every sale in a blink. Which error message have you seen on your phone, and what did it mean?

Before this session: DA02 Values, names and types. If you cannot explain the last example, use its error card with the instructor first.

Words for today

Word	What it means	Think of it like this
operator	A symbol that calculates: + - * / and **.	The buttons on a calculator.
expression	A calculation Python can work out, such as 3 * 2000.	One line of working in a maths exercise.
order of operations	Brackets first, then powers, then multiply and divide, then add and subtract.	The rule that makes 2 + 3 * 4 equal 14.
comment	A note for people that starts with # and is ignored by Python.	A pencil note in the margin.
error message	Python explaining why it stopped. Read the last line first.	A friend pointing out what went wrong.
line number	The place in the cell where Python found the problem.	The page and line in a book.

Block A. Revenue by hand and by Python

Revenue is units times unit price. The first three sales in the shop table are 1 Book at 3000, 2 Pens at 1000 and 4 Juice at 2000. Work the three revenues out on paper first. A line that starts with # is a comment.

Predict. Predict the three revenues and their total, using paper.

Your guess, written before you run it

```
# revenue = units * price
book = 1 * 3000
pen = 2 * 1000
juice = 4 * 2000
```

```
total = book + pen + juice
print(book, pen, juice)
print(total)
print(2 + 3 * 4)
print((2 + 3) * 4)
```

Run it, then compare. The cell prints 3000 2000 8000, then 13000, then 14 and 20. Multiplication happens before addition. Brackets change the order.

Investigate.

1. Why does 2 + 3 * 4 give 14 and not 20?
2. Why do we say revenue and not profit?

Modify. Change the Pen to 5 units and find the new total.

Block B. Reading an error message

An error is feedback about an instruction. It is not a judgement about you. Read the last line first. It names the kind of error. Then find the line number it points to. Type this line in a new cell and run it: print(10 / 0). Then type print(units) in another cell before you have created units.

Predict. Predict the name of each error before you run the lines. Write your guess on paper first.

Your guess, written before you run it

Run it, then compare. The first line stops with ZeroDivisionError: division by zero, because no number answers 10 divided by 0. The second stops with NameError: name 'units' is not defined, because Python has not met a line that creates units. In both cases the last line of the message says what went wrong.

Investigate.

1. Which line of the message do you read first, and why?
2. How do you fix a NameError?

Modify. Fix the NameError by creating units = 4 first and then printing units * 2000.

Make

Convert the text "12.5" to a number and calculate the price of four items at that price. Predict the result before you run it. Write one sentence on why division by zero cannot be answered.

Save your work as DA03. Keep the worked example and put your own attempt below it. Write two sentences: what you changed and how you checked it.

Error card

You see	It means	Fix
ZeroDivisionError: division by zero	You divided by zero, which has no answer.	Check the number you divide by. Change it or guard against zero in a later session.
NameError: name 'units' is not defined	The name was never created in this session or is spelled differently.	Run the cell that creates it and compare the spelling letter by letter.

Exit ticket

I read the last line of an error first, find its line number, and fix one thing at a time.

AI use follows AI_USE_POLICY.pdf. In weeks 1 to 6 no AI-generated solutions are used for exercises, labs or build tasks. Practise for 20 to 30 minutes between classes by repeating the Make task with one value changed. If you are stuck for 30 minutes, save the error and bring it to class. A file saved on your laptop has not been shared until you use the route in START_HERE.pdf.